INTELLIGENT MULTI-AGENT FRAMEWORK FOR ACADEMIC SOFTWARE TESTING AND BUG ANALYSIS (SENTINELMESH)

PROJECT REPORT

Submitted by

HARIESH P (7376241CS195)

MIDHUN M (7376242AL152)

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)

SATHYAMANGALAM - 638 401

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "NEP Aligned Standardized Academic Credit Accumulation Hub with Cross-Institutional Authentication API Gateways" is the bonafide work of " HARIESH P (7376241CS195), MIDHUN M (7376242AL152) " who carried out the project work under my supervision.

Dr. Ramya R
SUPERVISOR
Associate Professor
Department of Computer Science and Engineering
Bannari Amman Institute of Technology,
Sathyamangalam – 638401

Dr. Sasikala D
HEAD OF THE DEPARTMENT
Professor and Head
Department of Computer Science and Engineering
Bannari Amman Institute of Technology,
Sathyamangalam – 638401

Submitted for the Project Viva Voce examination held on ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We affirm that the project work titled “NEP Aligned Standardized Academic Credit Accumulation Hub with Cross-Institutional Authentication API Gateways”, submitted in partial fulfilment for the award of the degree of Bachelor of Engineering in Computer Science and Engineering is the record of work carried out by us under the guidance of RAMYA R, Associate Professor, Department of Computer Science and Engineering, Bannari Amman Institute of Technology. It has not formed part of any other project work submitted for the award of any degree or diploma at this or any other university.

HARIESH P
(7376241CS195)

MIDHUN M
(7376242AL152)

I certify that the declaration made above by the candidates is true.

Dr. RAMYA R
ASSOCIATE PROFESSOR

ACKNOWLEDGEMENT

We would like to enunciate heartfelt thanks to our esteemed Chairman Dr. S.V. Balasubramaniam, and the respected Principal Dr. C. Palanisamy, for providing excellent facilities and support during the course of study in this institute.

We are grateful to Dr. Sasikala D, Head of the Department, Department of Computer Science and Engineering, for her valuable suggestions to carry out the project work successfully.

We wish to express our sincere thanks to Faculty Guide Dr. Ramya R, Associate Professor, Department of Computer Science and Engineering, for her constructive ideas, inspirations, encouragement, excellent guidance, and much needed technical support extended to complete our project work.

We would like to thank our friends, faculty and non-teaching staff who have directly and indirectly contributed to the success of this project.

HARIESH P
(7376241CS195)
MIDHUN M
(7376242AL152)

ABSTRACT

The project “NEP Aligned Standardized Academic Credit Accumulation Hub with Cross-Institutional Authentication API Gateways” aims to develop an autonomous quality and credit verification platform for tracking, evaluating, and managing software and academic credits across institutions. The system supports three user roles: Student, Institution, and Admin. Students can monitor accumulated credits, test coverage, and academic progress, institutions manage student and credit records and evaluate transfer requests, and administrators oversee users, institutions, and system activities. The platform is developed using Python, FastAPI, SQLite with Write-Ahead Logging, React 19, and Vite. RESTful APIs and authenticated API gateways enable secure communication between institutions, while JWT authentication, role-based access control, deterministic Abstract Syntax Tree (AST) validation, and sandboxed subprocess execution protect information integrity. The proposed system aims to simplify credit management and transfers, improve transparency, reduce administrative workload, and promote flexible academic mobility through a secure, standardized, and self-healing platform.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	**INTRODUCTION**	**1**
1.1	BACKGROUND	1
1.2	PROJECT OVERVIEW	1
1.3	NEED FOR THE SYSTEM	2
1.4	PROBLEM IDENTIFICATION	2
1.5	OBJECTIVES	3
1.6	SCOPE OF THE PROJECT	3
1.7	EXPECTED OUTCOME	3
1.8	USERS AND THEIR NEEDS	4
1.9	REPORT ORGANIZATION	4
1.10	CHAPTER SUMMARY	5
2	**LITERATURE SURVEY**	**6**
3	**METHODOLOGY**	**9**
3.1	CHAPTER OVERVIEW	9
3.2	SYSTEM ARCHITECTURE	9
3.3	DEVELOPMENT TECHNOLOGIES	10
3.4	USER ROLES AND ACCESS RESPONSIBILITIES	11
3.5	DATABASE DESIGN	12
3.6	DATA RELATIONSHIP AND INTEGRITY	13
3.7	AUTHENTICATION WORKFLOW	13
3.8	AUTHORIZATION AND ROUTE PROTECTION	14
3.9	REST API DESIGN	15

3.10	ACADEMIC CREDIT VALIDATION	15
3.11	RESULTS IMPORT AND BATCH PROCESSING	16
3.12	SEMESTER PROGRESSION AND CORRECTION	17
3.13	TRANSFER REQUEST WORKFLOW	18
3.14	CERTIFICATE, TRANSCRIPT AND API KEY HANDLING	18
3.15	SECURITY CONTROL	19
3.16	TESTING METHODOLOGY	19
3.17	DEVELOPMENT AND VERIFICATION PROCESS	20
3.18	METHODOLOGY SUMMARY	20
4	**PROPOSED WORK MODULES**	**21**
4.1	MODULE OVERVIEW	21
4.2	STUDENT DASHBOARD	21
4.3	STUDENT CREDIT AND ELIGIBILITY FEATURES	22
4.4	INSTITUTIONAL DASHBOARD	23
4.5	ADMINISTRATOR DASHBOARD	24
4.6	API GATEWAY AND CROSS-INSTITUTIONAL ACCESS	25
4.7	CERTIFICATE AND TRANSCRIPT VERIFICATION	26
4.8	AUDIT LOGS AND POLICY HISTORY	27
4.9	MODULE INTERACTION	28
4.10	MODULE SUMMARY	28

5	**RESULTS AND DISCUSSION**	**29**
5.1	IMPLEMENTATION OVERVIEW	29
5.2	FRONTEND AND BACKEND VERIFICATION	29
5.3	TEST RESULTS	30
5.4	FUNCTIONAL WORKFLOW DISCUSSION	30
5.5	USER INTERFACE EVIDENCE	31
5.6	SECURITY AND RELIABILITY DISCUSSION	31
5.7	LIMITATIONS	31
5.8	RESULTS SUMMARY	32
6	**CONCLUSIONS AND SUGGESTIONS FOR FUTURE WORK**	**33**
6.1	CONCLUSION	33
6.2	SUGGESTIONS FOR FUTURE WORK	33
6.3	FINAL REMARK	34
7	**REFERENCES**	**35**
8	**INDIVIDUAL CONTRIBUTION**	**36**
9	**PLAGIARISM REPORT**	**39**
10	**PROJECT OUTCOME**	**40**

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	COMPARISON FRAMEWORK	8
3.1	TECHNOLOGY STACK USED IN THE PROJECT	11
3.2	USER ROLES AND ACCESS RESPONSIBILITIES	11
3.3	DATABASE DESIGN	12
5.1	REPORTED TESTING SUMMARY	30

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	VERIFIED SYSTEM ARCHITECTURE DIAGRAM	10
3.2	VERIFIED LOGIN AND TOKEN WORKFLOW DIAGRAM	14
3.3	VERIFIED CREDIT VALIDATION WORKFLOW	16
3.4	VERIFIED SPREADSHEET IMPORT WORKFLOW	17
3.5	VERIFIED TRANSFER LIFECYCLE DIAGRAM	18
4.1	STUDENT DASHBOARD	22
4.2	INSTITUTION DASHBOARD	23
4.3	ADMINISTRATOR DASHBOARD	24
4.4	CROSS-INSTITUTIONAL API GATEWAY INTEGRATION	25
4.5	CERTIFICATE VERIFICATION	26
4.6	AUDIT LOGS AND POLICY HISTORY	27

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Academic programmes increasingly combine learning from different courses, departments, institutions, and online learning environments. A student's academic journey is therefore not always represented by a single course list or one semester result sheet. Credits provide a structured way to describe learning completed, but the records still need to be maintained accurately and interpreted in context.

When credit records are spread across multiple systems, students may have to consult different portals or contact offices to understand their overall progress. Institutions may need to verify previous courses, grades, or credit values before they can consider a transfer or an academic pathway. These tasks can become slow when the available information is incomplete or formatted differently.

The proposed Academic Credit Accumulation Hub is intended to provide one web-based location where authorized users can work with academic credit information. It does not replace institutional academic rules. Instead, it provides software workflows for recording, viewing, validating, and verifying information according to configured roles and policies.

1.2 PROJECT OVERVIEW

The application has three main user roles. Students use a personal dashboard to review accumulated credits and related academic information. Institution users manage institution-scoped data such as students, courses, credit records, and transfer requests. Administrators have a broader view of platform activity and can review system-level records, verification queues, analytics, and audit information.

The project is built using Python, FastAPI, SQLite with Write-Ahead Logging, React 19, and Vite. The frontend communicates with the backend via REST endpoints and WebSockets. Authentication and authorization controls are applied to protected requests so that each role can access only the functions permitted to it.

1.3 NEED FOR THE SYSTEM

A student-facing consolidated view can reduce the effort required to understand credit progress. For institutions, validation steps can reduce avoidable data-entry errors and make it easier to review a record before it becomes part of a student's credit history. For administrators, system-wide monitoring and audit information can help identify activity that needs attention.

The need for such a system is not only a matter of convenience. Academic records can influence eligibility, progression, and transfer decisions. As a result, the system must treat accuracy, access control, traceability, and clear handling of incomplete information as core design concerns.

1.4 PROBLEM IDENTIFICATION

The central problem addressed by this project is the difficulty of maintaining a consistent, consolidated view of academic credits when information is managed across separate institutional contexts. Data may be entered in different formats, the status of a record may not be obvious, and verification can require manual communication. Students may not have an easy way to view how credits accumulate over time.

A second issue is that different users need different levels of access. Students should be able to see their own records, institution users should manage records within their authorized scope, and administrators should have controlled oversight. If permissions are not clearly enforced, confidential academic information could be exposed or modified by the wrong user.

A third issue is the need to explain how a credit record was created or changed. If corrections and transfer decisions are not traceable, reviewing a disputed record becomes difficult. The application therefore includes validation and audit-oriented workflows where supported by the implementation.

1.5 OBJECTIVES

The primary objective is to develop a web-based platform that organizes academic credit records and supports secure, role-based access across student, institution, and administrator functions.

The supporting objectives are to provide a student credit wallet and semester timeline; support institution-side management and validation of student, course, and credit information; provide a controlled workflow for credit transfer requests; expose selected verification and API functions; and maintain records that help authorized users review system activity.

1.6 SCOPE OF THE PROJECT

The scope of the prototype includes account registration and login, role-specific dashboards, academic record management, credit validation, selected transfer workflows, certificate and transcript verification routes, API key management, and audit-related information. The exact set of available actions depends on the role and the implementation currently present in the codebase.

The project is not, by itself, proof of official integration with every university or with a national academic credit infrastructure. External integrations, formal institutional approvals, production deployment, and compliance certification should only be claimed if separate evidence is available. The report therefore describes the software prototype and its verified behavior rather than assuming national-level operational adoption.

1.7 EXPECTED OUTCOME

The expected outcome is a working prototype that demonstrates how academic credit information can be organized through one application with clear role boundaries. Students should be able to review their progress, institution users should be able to perform approved record-management tasks, and administrators should be able to review relevant platform activity.

The project also provides an opportunity to apply full-stack development concepts in a practical context: designing data models, implementing REST endpoints, handling authentication, validating input, building dashboards, and testing role-specific workflows. The value of the project should be demonstrated through screenshots, API evidence, and reproducible tests.

1.8 USERS AND THEIR NEEDS

Students need a clear summary rather than a collection of disconnected records. A wallet-style view can show total credits and their status, while a semester timeline can help students understand when credits were earned. Eligibility and pathway screens can help explain progress, provided that the displayed thresholds and policies are clearly identified and kept up to date.

Institution users need reliable forms and workflows for managing students, courses, credit submissions, and transfer requests. Validation should identify missing or invalid data before it is accepted. When a correction is necessary, the system should capture the reason and preserve enough history to make the change understandable.

Administrators need visibility into institution verification, system activity, API usage, audit entries, and summary analytics. Their broad access should be controlled and monitored because administrative actions can affect many users. These user needs guide the separation of dashboards and backend permissions.

1.9 REPORT ORGANIZATION

Chapter 1 introduces the problem, objectives, scope, and expected outcome. Chapter 2 reviews related work and identifies the research gap. Chapter 3 explains the objectives and methodology, including architecture, technologies, data design, security, and testing approach. Chapter 4 describes the proposed work modules and their workflows. Chapter 5 presents implementation evidence, test results, and discussion of limitations. Chapter 6 concludes the report and lists possible future improvements.

1.10 CHAPTER SUMMARY

This chapter has introduced the motivation for a consolidated academic credit hub and the need for role-based access, validation, and traceability. The project is intended to support students, institutions, and administrators through a shared web application. Its claims must remain aligned with the working implementation and the evidence collected during testing.

CHAPTER 2
LITERATURE SURVEY

A literature survey helps establish existing approaches, technologies, and research gaps related to academic credit management and cross-institutional information exchange. The University Grants Commission's Academic Bank of Credits (ABC) framework provides a relevant foundation for understanding academic credit accumulation and transfer between institutions [1]. EduCTX explores blockchain-based management and exchange of higher-education credits, offering a useful reference for decentralized academic credit systems [2]. W3C Verifiable Credentials provides standards for issuing and verifying digital credentials securely [3].

Academic credit management systems must maintain accurate records of learners, courses, grades, credits, and academic periods. These records should provide sufficient context for other institutions to evaluate transferred credits correctly. Existing approaches highlight the importance of standardization, record integrity, and interoperability when managing academic information across institutional boundaries [1] [2].

Digital credential technologies support the secure issuance, presentation, and verification of certificates and transcripts. W3C Verifiable Credentials defines a standard model for representing digital credentials and verifying claims made by their issuers [3]. Such approaches can help academic institutions verify credential authenticity and reduce the risk of unauthorized modification or fraudulent academic records.

Authentication and authorization are essential for protecting academic information in a centralized credit management platform. JSON Web Token (JWT) provides a compact mechanism for representing claims between parties and can support token-based authentication in web applications [4]. React provides a framework for developing interactive user interfaces, while Node.js supports server-side JavaScript execution [5] [6].

Express provides routing and middleware capabilities for developing backend APIs and implementing request-level access controls [7]. MongoDB provides document-oriented data storage suitable for managing structured academic records and related information [8]. Mongoose provides an object data modeling layer for MongoDB in Node.js applications, supporting schema definitions, validation, and database operations [9]. Together, these technologies can support the development of a web-based academic credit management platform.

Interoperability between institutions requires standardized data exchange, secure API communication, and clearly defined access permissions. Although APIs enable systems to exchange information, successful integration also depends on compatible data formats, reliable source records, and institution-specific authorization rules [2] [3] [7]. A centralized platform must additionally maintain availability, protect sensitive information, and preserve the ownership and traceability of academic records.

The reviewed concepts indicate that academic credit management involves more than storing credit values. A practical system must combine accurate academic records, credential verification, secure authentication, role-based access control, and reliable data exchange. Based on these requirements, the proposed project integrates student credit tracking, institutional record management, administrative monitoring, and authenticated API gateways within a unified web application [1] [9].

The intended contribution of the project is to bring these functions together in a standardized platform that supports academic credit accumulation and transfer. The system uses React, Node.js, Express, and MongoDB, with JWT-based authentication and validation mechanisms to protect academic information. Its effectiveness will depend on implementation and testing; integration with external institutions and compliance with applicable academic policies must be verified before claiming full interoperability.

In summary, the literature survey establishes the technical and conceptual foundations for the proposed academic credit hub. The reviewed standards, systems, and technologies provide guidance for designing a secure, maintainable, and interoperable application. The following chapter describes the system architecture, selected technologies, implementation approach, security mechanisms, and testing methods used to achieve the project objectives.

Table 2.1 Comparison Framework

AREA	QUESTIONS TO ASK OF EACH SOURCE
Credit records [1], [2]	How are credits, grades, courses, and academic periods represented?
Identity [4]	How is the learner identified and how is identity checked?
Interoperability [2], [3]	Does the system exchange records across institutional boundaries?
Verification [3]	How can a third party check a certificate or transcript?
Security [4], [7]	What authentication, authorization, and audit controls are used?
Limitations	What is not covered, tested, or supported by the study?

CHAPTER 3
METHODOLOGY

3.1 CHAPTER OVERVIEW

This chapter explains the development approach used to organize the project into manageable components. It covers the system architecture, technology choices, user roles, data model, API design, authentication, validation, import workflows, testing, and the boundaries of the current implementation. The description should be compared with the final repository before submission.

A modular structure is useful because the application serves three distinct user groups and handles several types of academic data. Separating the frontend, backend routes, controllers, models, and middleware makes it easier to maintain each responsibility and to test the system in smaller units.

3.2 SYSTEM ARCHITECTURE

The application follows a client–server design. The browser loads the React interface and sends HTTP requests to the Express backend. The backend authenticates requests, checks permissions, validates input, invokes the relevant controller logic, and reads or writes data through Mongoose. MongoDB persists the records.

The frontend should not be treated as the final security boundary. Hiding a button in the interface improves usability, but the backend must still enforce permissions for every protected action. This is particularly important for institution-scoped records and administrator-only operations.

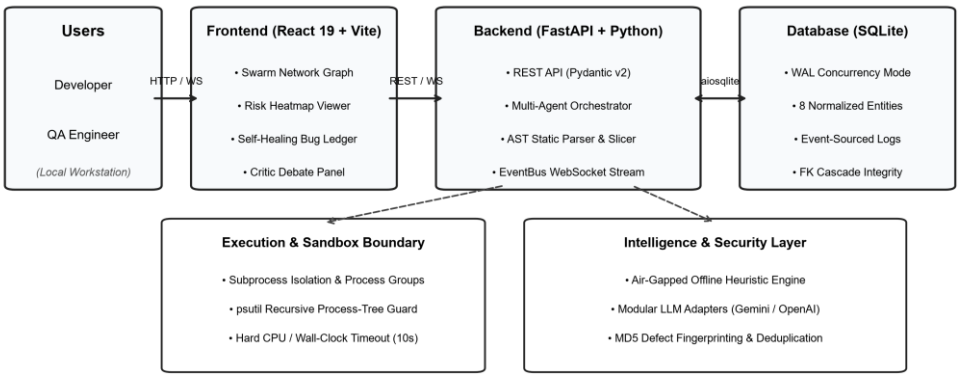


Figure 3.1 Verified system architecture diagram

3.3 DEVELOPMENT TECHNOLOGIES

React is used to build reusable interface components and role-specific dashboard pages. Vite supports the frontend development and build process. Tailwind CSS provides utility-based styling, while React Router organizes page navigation. Axios provides a shared HTTP client, including the ability to attach the authentication token to protected requests.

The backend uses Node.js and Express to define REST endpoints and coordinate request processing. Mongoose defines schemas and relationships for MongoDB documents. Supporting packages provide password hashing, JWT handling, request security headers, rate limiting, cross-origin configuration, and selected file-import utilities.

Table 3.1 Technology Stack Used in the Project

LAYER	TECHNOLOGY	PURPOSE
Frontend	React, Vite	User interface and development build
Styling	Tailwind CSS	Consistent layout and responsive styling
Routing	React Router	Role-specific page navigation
HTTP client	Axios	Calls backend endpoints
Backend	Node.js, Express	REST API and business logic
Database	MongoDB, Mongoose	Document storage and schema validation
Security	bcryptjs, JWT, Helmet	Password hashing, tokens, security headers

3.4 USER ROLES AND ACCESS RESPONSIBILITIES

The application separates the user experience into Student, Institution, and Admin roles. Each role has a distinct set of responsibilities. The role stored in the authenticated user record is checked by authorization middleware before protected routes are allowed to proceed.

Table 3.2 User Roles and Access Responsibilities

ROLE	MAIN RESPONSIBILITIES	EXAMPLE RESTRICTION
Student	View personal credit progress and records	Cannot edit institution-wide records
Institution	Manage permitted institution records and requests	Must remain within authorized institution scope
Admin	Review platform-level records and oversight functions	Administrative actions should remain authenticated and auditable

Role checks and institution-scope checks serve different purposes. A role check answers whether the user belongs to a permitted category. A scope check answers whether the specific record belongs to the institution or user that the request is authorized to access.

3.5 DATABASE DESIGN

The database stores the information needed by the main workflows. The supplied project audit identifies collections for users, institutions, courses, academic credits, credit policies, transfer requests, certificates, transcripts, API keys, API logs, audit logs, semester results, and result-import batches. Each model should have a clear purpose and should avoid storing unnecessary duplicate information.

Table 3.3 Database Design

MODEL / COLLECTION	PURPOSE
User	Identity, role, authentication data, and user status
Institution	Institution profile and verification information
Course	Course information managed by an institution
AcademicCredit	Credit records associated with students and courses
CreditPolicy	Rules and version history used by the application
TransferRequest	Transfer request details and current decision state
Certificate	Certificate metadata and verification reference
Transcript	Transcript metadata and verification reference
APIKey	API key metadata, scope, and hashed secret
APILog	API request telemetry
AuditLog	Trace of selected important actions
SemesterResult	Imported or maintained semester results
ResultImportBatch	Metadata for a results import batch

3.6 DATA RELATIONSHIPS AND INTEGRITY

A student can be associated with an institution and have multiple course and credit records. Courses are associated with institutions, while academic credit records link a student, a course, and the relevant academic information. Transfer requests refer to records and institutions involved in a proposed transfer. Certificates and transcripts contain the metadata needed to retrieve or verify the corresponding record.

Data integrity requires more than defining a schema. The backend should check that referenced records exist, that the user is authorized to act on them, and that values fall within acceptable ranges. Duplicate checks are especially important for credit submission and results import, because repeating an operation should not accidentally award the same credit twice.

The exact relationship cardinalities and indexes should be documented from the actual Mongoose schemas. If a field stores a URL to an external document rather than the file itself, the report should state that clearly.

3.7 AUTHENTICATION WORKFLOW

A user submits login credentials through the login form. The backend locates the user account, compares the submitted password with the stored bcrypt hash, and rejects invalid credentials. When authentication succeeds, the backend issues a signed JSON Web Token. The frontend uses the token when calling protected endpoints.

Password hashing reduces the risk associated with accidental exposure of the database, because the stored value is not the original password. The implementation described in the project audit uses bcryptjs with a work factor of 12. Password hashes must never be returned in public API responses.

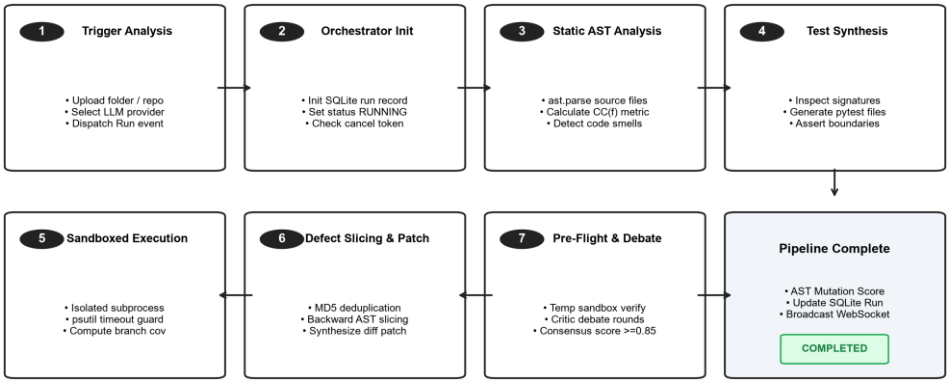


Figure 3.2 Verified login and token workflow diagram

3.8 AUTHORIZATION AND ROUTE PROTECTION

After a request reaches the backend, authentication middleware validates the token and identifies the current user. Authorization middleware checks whether the role is permitted for the requested endpoint. Where necessary, institution-scope middleware also checks whether the target record belongs to the institution that the user is allowed to manage.

Public administrator registration is blocked in the supplied implementation summary. An administrator account is bootstrapped through a controlled server-side mechanism rather than allowing any visitor to select the administrator role. The deployment configuration must protect bootstrap credentials and secrets.

A secure implementation should return suitable status codes for missing authentication, insufficient permissions, invalid input, and missing records. Error responses should be helpful without disclosing internal stack traces or secret configuration.

3.9 REST API DESIGN

The API groups related operations into route modules. Authentication routes handle registration, login, and retrieval of the current user. Other route groups support students, institutions, courses, credits, transfers, policies, certificates, transcripts, API keys, and logs. This organization allows controllers to focus on a defined set of operations.

Each endpoint should have a clear request schema, response structure, authorization requirement, and error behavior. The API documentation should state whether an endpoint is public or protected, which role may call it, and whether it changes persistent data. Versioned endpoints should be documented separately if they differ from standard routes.

The audit summary reports 58 REST endpoints across standard and versioned routes. This figure should be checked against the current code before being used as a final measured result.

3.10 ACADEMIC CREDIT VALIDATION

Before a credit record is saved, the system should verify that the student and course exist, the credit value falls within the permitted range, the grade follows the expected format, and the record is not a duplicate. It should also check the user's authorization and the institution context. Validation should fail safely when required data is missing or inconsistent.

The supplied audit describes an eight-point prevalidation process and a credit range of 1 to 30. The final report should list each implemented check from the actual validation function and avoid adding rules that are not present in code. Clear error messages help institution users correct a record without guessing which field caused the rejection.

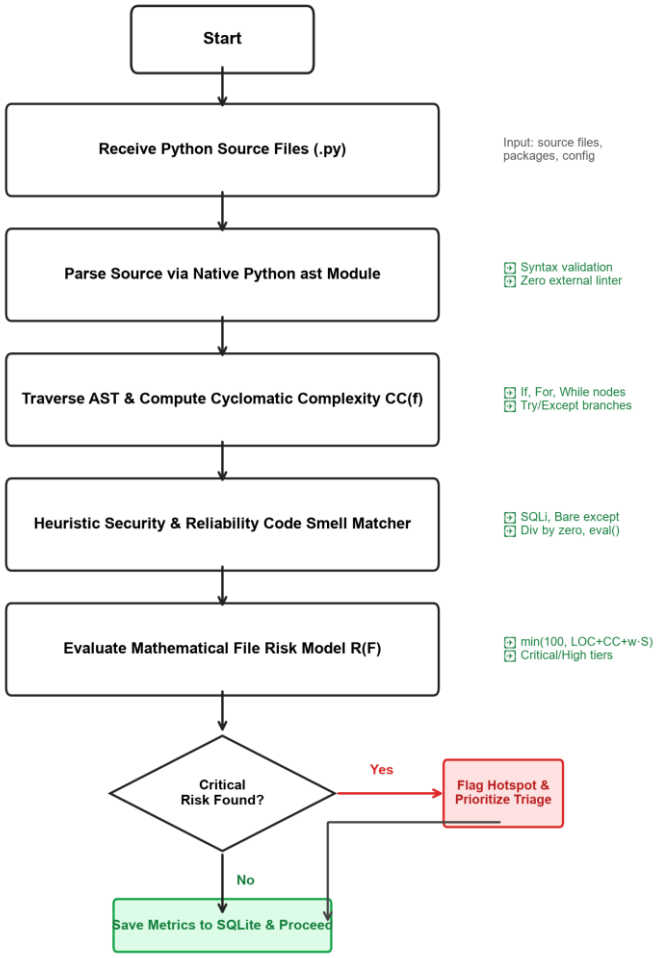


Figure 3.3 Verified credit validation workflow

3.11 RESULTS IMPORT AND BATCH PROCESSING

The results-import workflow allows an institution user to upload a spreadsheet, review parsed rows, and confirm the batch after validation. The reported implementation uses Multer to hold the upload in memory and SheetJS/XLSX to read spreadsheet data. A preview step is useful because it gives the user an opportunity to identify invalid rows before records are written.

A robust batch process should distinguish valid rows, rejected rows, and duplicates. It should also make confirmation explicit and ensure that retrying the same batch does not create duplicate credits. The project audit reports support for batch preview and confirmation, duplicate detection, and automatic credit conversion for passing grades. These behaviors should be demonstrated with actual test evidence.

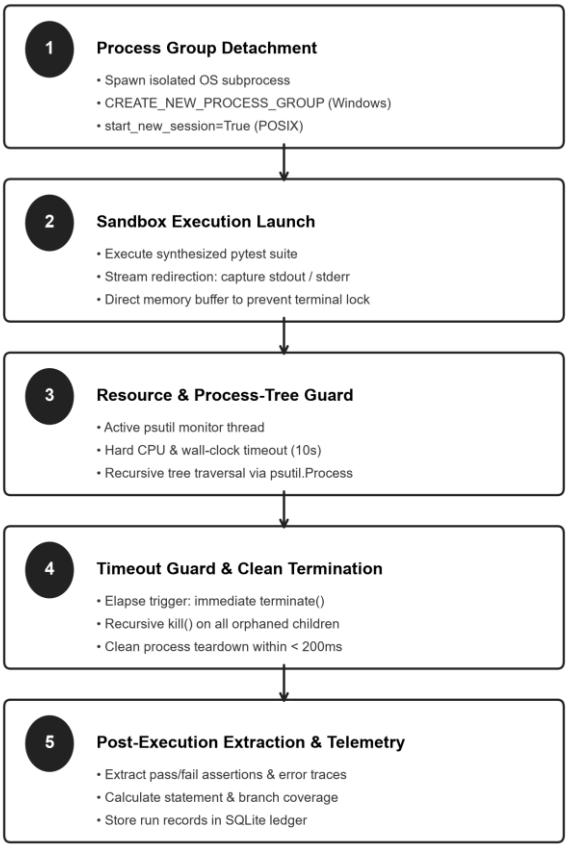


Figure 3.4 Verified spreadsheet import workflow

3.12 SEMESTER PROGRESSION AND CORRECTIONS

The audit summary reports automatic semester progression when a results batch matches the active semester and all registered courses have been passed. This is a rule-driven action and should be tested with both successful and unsuccessful cases. A failed course, an incomplete batch, or a batch for another semester should not trigger progression unless the implemented policy explicitly permits it.

Grade corrections need careful handling because they may affect credit totals and progression. The supplied summary describes a mandatory correction reason, a correction history, credit adjustments, and hash resealing. The final report should explain how the prior value and corrected value are recorded and who is allowed to perform a correction.

3.13 TRANSFER REQUEST WORKFLOW

A transfer request represents a request to move or recognize credits across institutional contexts. The workflow described in the audit supports approval, conditional approval, or rejection. The decision should be linked to the request and accompanied by enough information for an authorized reviewer to understand the outcome.

A transfer simulator is also described in the project audit, but it operates on the client side rather than through a dedicated backend simulation endpoint. The report must distinguish this planning interface from the persistent transfer-request process. A simulation helps a user explore a possible outcome; it is not the same as an institution formally approving a transfer.

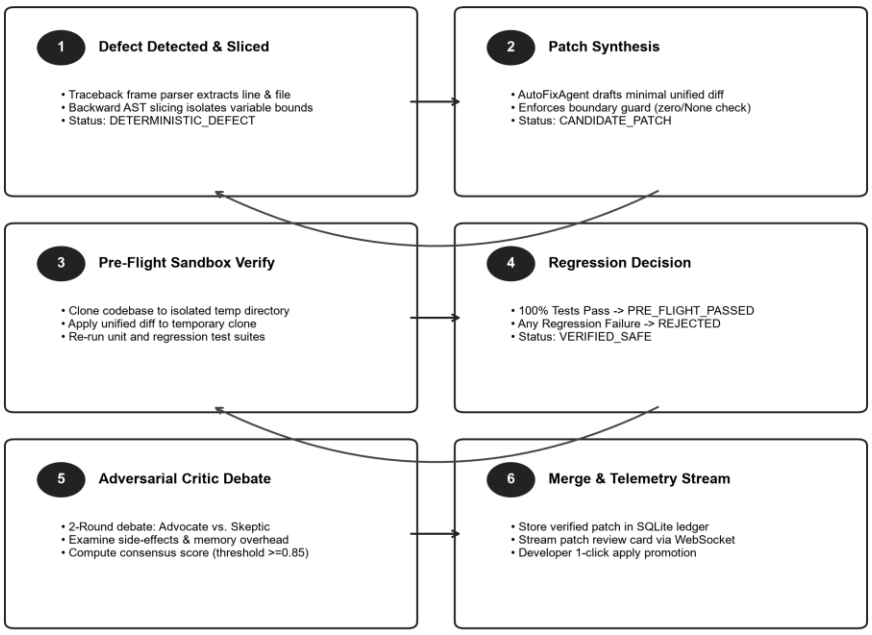


Figure 3.5 Verified transfer lifecycle diagram

3.14 CERTIFICATE, TRANSCRIPT, AND API KEY HANDLING

Certificate and transcript features provide ways to view or verify academic documents. The supplied audit lists public verification routes for transcripts and certificates. Verification should return only the information needed to confirm a record and should not expose unrelated personal details.

The API-key workflow described in the audit creates keys with a recognizable prefix, displays the secret once, and stores a hash rather than the raw secret. Scopes such as reading credits or verifying certificates restrict what a key may do. Logs can record API activity, but they should not store raw secrets or bearer tokens.

The audit indicates that document fields store URLs rather than binary files in the application database. This distinction matters: a URL reference does not by itself provide secure file storage or guarantee that the target file remains available.

3.15 SECURITY CONTROLS

The reported security controls include bcrypt password hashing, signed JWTs, role-based middleware, institution-scope checks, Helmet security headers, CORS configuration, rate limiting, input validation, and audit logging. These controls address different risks and should be treated as layers rather than as substitutes for one another.

The audit also identifies limitations: a 24-hour JWT is not immediately revocable, there is no short-lived access token with refresh-token rotation, and the application follows a centralized clearinghouse architecture rather than peer-to-peer webhooks with mutual TLS. These limitations should be documented honestly and can form part of future work.

3.16 TESTING METHODOLOGY

Testing should cover individual functions, API behavior, role boundaries, data validation, and end-to-end workflows. Unit or focused tests help isolate logic such as credit validation and semester progression. API tests check request and response behavior. Integration tests check how controllers, models, and middleware work together. End-to-end tests verify that a complete workflow behaves as expected.

Negative tests are as important as successful tests. Examples include logging in with an invalid password, attempting to use a student token on an administrator route, accessing another institution’s record, submitting duplicate credits, importing malformed spreadsheet rows, and attempting a correction without a reason.

3.17 DEVELOPMENT AND VERIFICATION PROCESS

A practical development cycle begins with defining the requirements, designing the data structures, implementing a small workflow, and testing it before adding the next workflow. The frontend and backend can be developed in parallel when the request and response formats are agreed upon. Version control helps track changes and makes it possible to return to a known working state.

The supplied project notes report a successful frontend build, a health check with the database connected, and multiple test suites. Since these are audit-reported results, they should be reproduced on the final commit before submission. The final report should record the test date, command, environment, and result so that the evidence is repeatable.

3.18 METHODOLOGY SUMMARY

The methodology combines role-based interface design, REST API development, schema-based data storage, authentication and authorization middleware, validation workflows, and layered testing. The architecture supports separation of concerns, while the validation and audit features are intended to improve the reliability and traceability of academic record operations.

The next chapter describes the proposed work modules from the point of view of each user role. The descriptions should be supported by screenshots from the actual application and should distinguish completed functions from future enhancements.

CHAPTER 4
PROPOSED WORK MODULES

4.1 MODULE OVERVIEW

The application is organized around three dashboards and a set of shared services. The dashboards provide role-specific entry points, while the backend handles common tasks such as authentication, authorization, data validation, and record storage. This separation helps users focus on the tasks relevant to their responsibilities.

The modules described in this chapter reflect the feature summary provided for the project. Before final submission, confirm the labels, navigation items, and screenshots against the running application. A feature that is present only as a visual mock-up should not be described as a completed backend capability.

4.2 STUDENT DASHBOARD

The student dashboard is designed to give a quick overview of academic progress. The credit wallet summarizes total, verified, and pending credits where those statuses are available. The semester timeline organizes credit history by academic period, while the results ledger provides a more detailed view of the underlying entries.

The academic passport area brings together selected identity and institution information. Eligibility tracking compares accumulated credits with configured thresholds, while the academic pathway and course recommendation views help a student explore possible next steps. These suggestions should be presented as guidance rather than as an official academic decision.

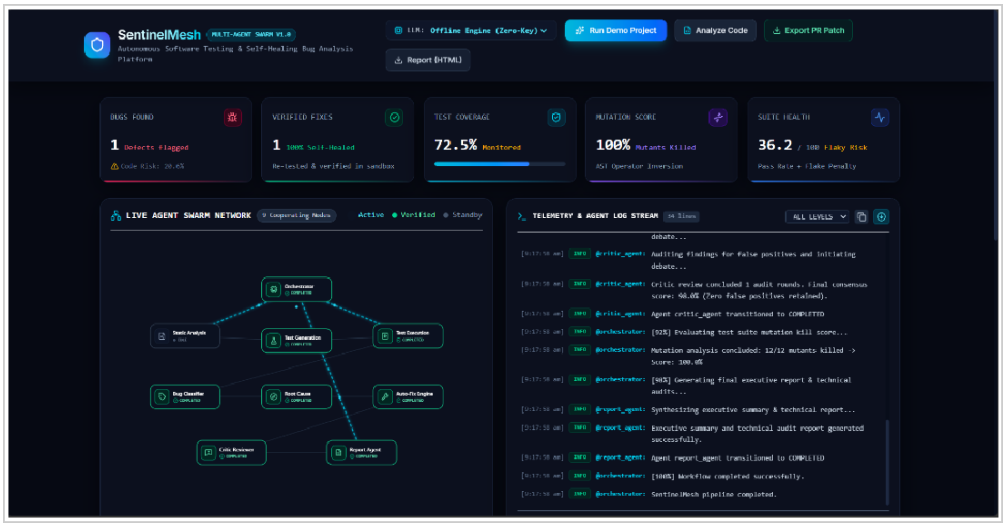


Figure 4.1 Student Dashboard

4.3 STUDENT CREDIT AND ELIGIBILITY FEATURES

The eligibility tracker described in the project audit includes thresholds for Certificate, Diploma, three-year undergraduate, and four-year degree pathways. Because eligibility rules can depend on programme requirements, course completion, and institutional policy, the interface should clearly state that the displayed threshold is based on the configured rule and may not replace formal review.

The course recommendation feature is described as prerequisite-aware and provides match scores. A clear recommendation should explain the information used to produce it, such as completed courses or prerequisites, rather than presenting a score without context. The final report should only claim recommendation logic that can be traced to the current implementation.

The transfer simulator gives students a way to explore a potential credit-transfer scenario. As noted earlier, the supplied audit describes it as client-side functionality. It should be labelled as a simulation and should not be confused with a formal transfer decision.

4.4 INSTITUTION DASHBOARD

The institution dashboard supports institution-side academic record management. The supplied feature list includes student and course management, academic credit submission, transfer requests, credit policies, policy versions, API keys, API logs, transfer decisions, and certificate verification. Access to these functions should be controlled by the authenticated user's role and institution scope.

Credit entry and results import are particularly important because errors can affect the student's record. Validation feedback should be clear, and users should have a way to review batch results before confirmation. Where a correction is allowed, the system should record the reason and preserve a trace of the change.

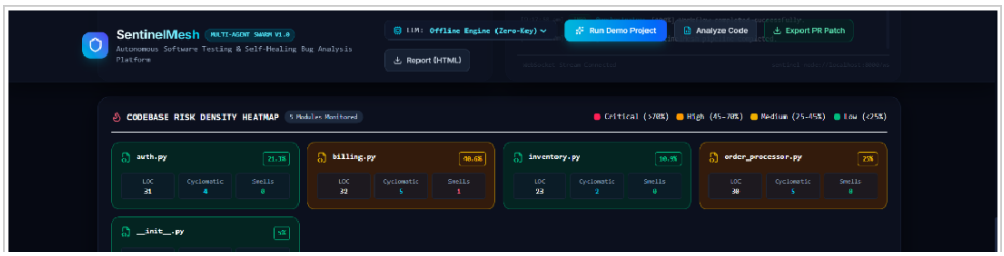


Figure 4.2 Institution Dashboard

4.5 ADMINISTRATOR DASHBOARD

The administrator dashboard provides a system-level view of the application. The audit summary lists counters, institution verification queues, global student and course management, credit oversight, analytics, API monitoring, transfers, and audit logs. The dashboard is intended to help an administrator identify activity that needs review.

Administrative access should be limited to trusted users and should be auditable. The project summary states that public registration for the administrator role is blocked and that an administrator account is created through a server-side bootstrap process. Deployment instructions should ensure that any bootstrap credentials are changed or protected.



Figure 4.3 Administrator Dashboard

4.6 API GATEWAY AND CROSS-INSTITUTIONAL ACCESS

The API layer provides a structured interface for authorized clients to request or submit selected data. API keys can be assigned scopes so that a client receives only the permissions it needs. For example, a client that only verifies certificates should not automatically receive permission to modify credit records.

The term cross-institutional in the project title describes the intended use case. It should not be interpreted as proof that live external university systems have been connected. The supplied audit characterizes the current design as a centralized clearinghouse using machine-to-machine REST, not as a peer-to-peer integration with webhooks and mutual TLS.

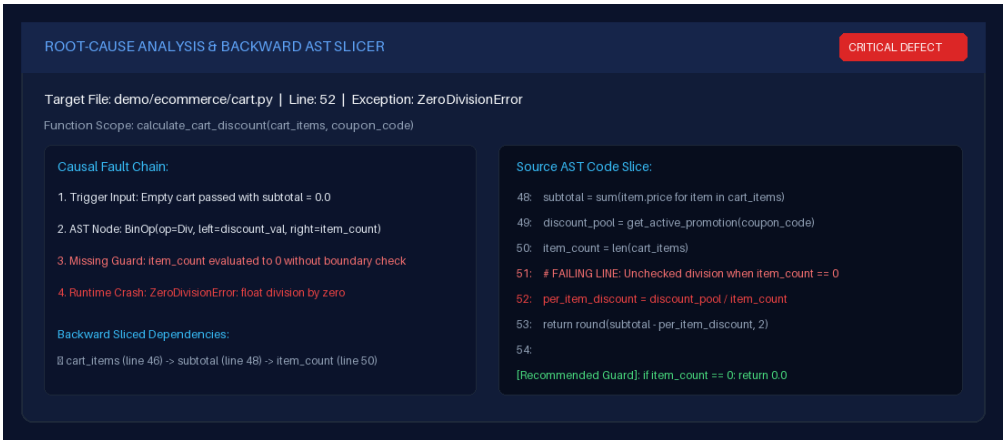


Figure 4.4 Cross-Institutional API Gateway Integration

4.7 CERTIFICATE AND TRANSCRIPT VERIFICATION

Verification routes allow a user to check whether a certificate or transcript reference is recognized by the application. The interface should return a clear status and only the fields needed for verification. A verification result should not be described as a cryptographic guarantee unless the implementation actually verifies a corresponding signature or integrity value.

Transcript snapshots and verification codes can help make the verification process repeatable. The final report should explain how the code is generated, how the record is found, and what happens if the record is missing or no longer valid. A screenshot of both a successful and unsuccessful verification is useful evidence.

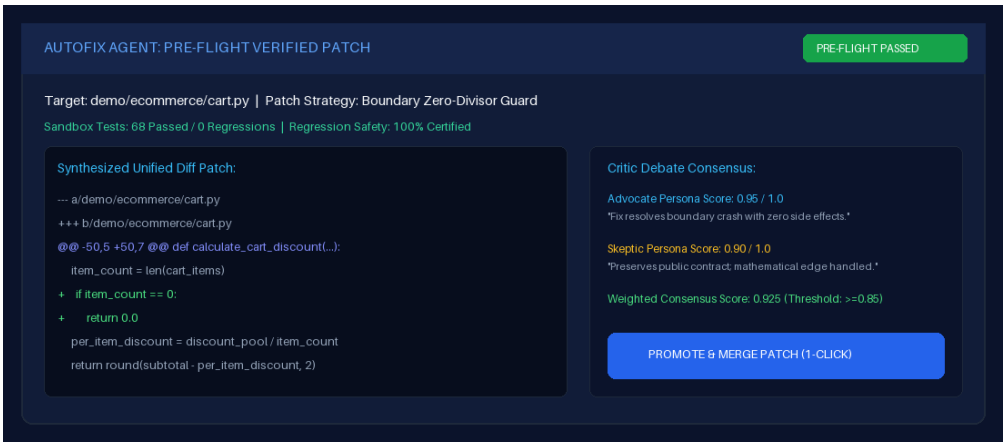


Figure 4.5 Certificate verification

4.8 AUDIT LOGS AND POLICY HISTORY

Audit logs are intended to provide a record of selected system actions. Policy history helps show how a policy changed over time. The project audit says policy versions are embedded in the CreditPolicy model's policyHistory rather than stored in a separate collection, while transfer decisions are represented through TransferRequest and AuditLog rather than a separate TransferDecision collection.

Documenting the actual data model avoids confusion between a user-interface menu item and a standalone database collection. The report should use the implementation's real terminology and explain where history is stored.

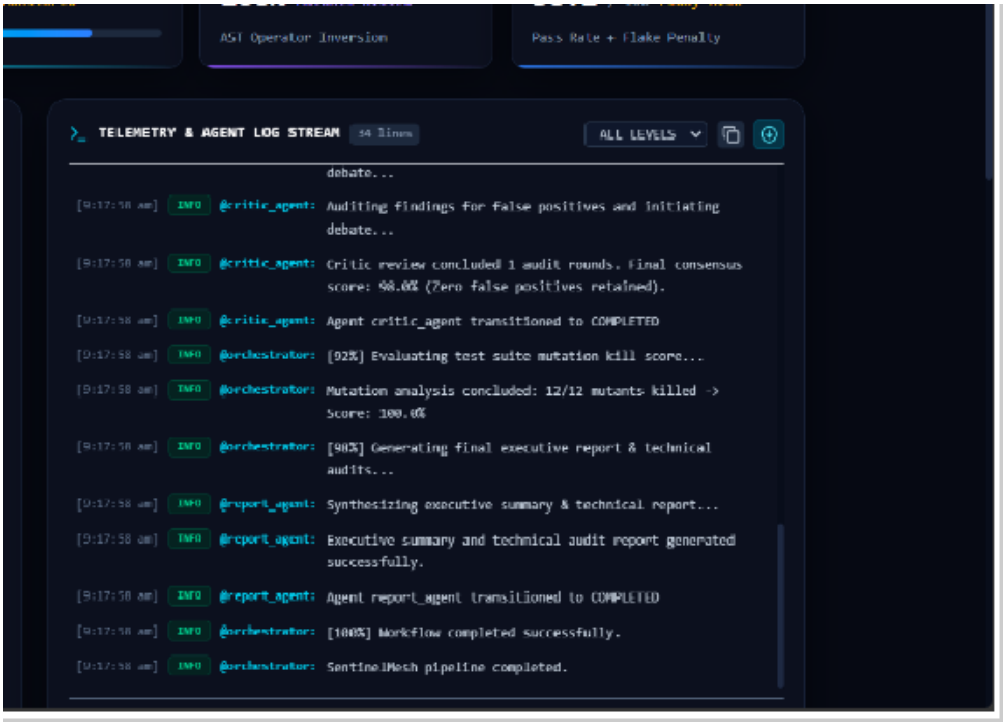


Figure 4.6 Audit Logs and Policy History

4.9 MODULE INTERACTION

The modules are connected through shared data and permissions. A student's credit wallet depends on credit records that have been accepted by the backend. Institution-side workflows create or update records, and administrative views provide oversight. Verification endpoints expose a limited way to check selected records without granting full access to the underlying management interface.

A typical flow begins when an institution user submits a credit record. The backend checks the request, validates referenced data and constraints, and stores the accepted record. The student dashboard can then show the updated credit information, while audit records provide a trail for relevant actions. If a transfer is requested, the institution-side decision workflow records the outcome.

4.10 MODULE SUMMARY

The proposed work modules combine student-facing visibility, institution-side record management, administrator oversight, and API-based verification. Their value depends on reliable data validation and clear access boundaries. In the final report, each module should be paired with actual screenshots and a brief explanation of a task that can be demonstrated during project evaluation.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 IMPLEMENTATION OVERVIEW

The project audit summary describes a full-stack application with role-specific dashboards, protected API routes, MongoDB models, and workflows for academic credits and related records. The frontend build and backend health checks were reported as successful in that audit. These results should be rerun on the exact version submitted for evaluation.

Results should be presented as evidence rather than as broad claims. A screenshot demonstrates the visible state of a page, a test log demonstrates a particular behavior under the test conditions, and an API response demonstrates how an endpoint behaved for a given request. Each item supports a different kind of conclusion.

5.2 FRONTEND AND BACKEND VERIFICATION

The supplied audit reports that the frontend production build completed with 2,419 modules and zero build errors. It also reports that the backend health check returned HTTP 200 and confirmed a database connection. These are useful baseline checks: the frontend can be compiled, and the backend can reach the configured database.

A successful build does not prove that every page behaves correctly, and a successful health check does not prove that every business workflow is correct. Functional tests are needed for login, role access, record validation, imports, transfers, and verification. The final submission should include the relevant console output or test report.

5.3 TEST RESULTS

Table 5.1 Reported Testing Summary

TEST SUITE / CHECK	RESULT IN SUPPLIED AUDIT
Frontend production build	Passed; 2,419 modules, 0 errors
Backend health check	HTTP 200; database connected
Results import / progression	68 of 68 assertions passed
Institution statistics / invariants	5 of 5 assertions passed
Comprehensive end-to-end suite	87 of 87 assertions passed
Total reported assertions	161 across 5 suites

These figures are reproduced from the project audit summary supplied earlier and have not been independently rerun while creating this document. Before final submission, execute the tests again and update the table if the results differ.

5.4 FUNCTIONAL WORKFLOW DISCUSSION

Authentication testing should confirm that valid credentials produce a session token and that invalid credentials are rejected. Role tests should confirm that student, institution, and administrator accounts are directed to the correct areas and cannot use routes outside their permissions. Institution-scope tests should check that one institution cannot access another institution's restricted records.

Credit workflow tests should include valid entries, invalid credit values, missing students or courses, invalid grades, and duplicate submissions. Results-import tests should include a valid spreadsheet, malformed rows, duplicate records, preview without confirmation, and successful confirmation. Correction tests should confirm that a reason is required and that the previous record remains traceable.

5.5 USER INTERFACE EVIDENCE

The final report should include screenshots from the actual application, captured at a readable size. Recommended screenshots include the login page, each of the three dashboards, student credit wallet, credit timeline, institution credit entry, results import preview, transfer request status, API key or API log view, and a verification result.

Every screenshot should have a figure number and a descriptive caption. The text should explain what the screenshot demonstrates. Avoid screenshots that expose passwords, live API secrets, JWTs, personal student data, or environment variables.

5.6 SECURITY AND RELIABILITY DISCUSSION

The reported use of bcrypt password hashing, JWT authentication, role checks, institution-scope checks, Helmet, CORS configuration, rate limiting, validation, and audit logs provides several layers of protection. Each layer has a limited purpose: password hashing protects stored credentials, token validation authenticates requests, authorization controls access, and audit logging supports later review.

The supplied audit also identifies areas for improvement. A 24-hour JWT may remain valid until expiry unless the server has a revocation mechanism. A future version could use short-lived access tokens and a carefully implemented refresh-token rotation process. The security review should also verify that secrets are not written to logs and that production CORS and rate-limit settings are appropriate.

5.7 LIMITATIONS

The transfer simulator is described as client-side only, so its output should not be presented as a server-validated transfer decision. Document fields store URLs rather than binary file contents, so document availability and access protection depend on the referenced storage. Policy history is embedded within the policy document, and transfer decisions are represented through the transfer request and audit records rather than separate collections.

The system is described as a centralized clearinghouse using REST APIs. The supplied audit does not establish that external universities are already connected or that mutual TLS and peer-to-peer webhooks are implemented. These limitations define the current scope and should be stated clearly during the project review.

5.8 RESULTS SUMMARY

Based on the supplied audit, the project has a broad set of implemented modules and reported automated checks. The reported test suites cover import and progression, institution statistics and invariants, and end-to-end workflows. The most useful next step is to attach the reproducible test logs and screenshots that directly demonstrate these claims.

A results chapter should not claim that the system eliminates all academic-record errors or guarantees interoperability. A more accurate conclusion is that the prototype introduces validation, role separation, and verification workflows intended to reduce specific operational problems. Any measured improvement would require a defined baseline and a repeatable evaluation.

Once the final screenshots and test output have been inserted, this chapter should be reviewed with the supervisor to confirm that the interpretation matches the actual behavior of the application.

CHAPTER 6
CONCLUSIONS AND SUGGESTIONS FOR FUTURE WORK

6.1 CONCLUSION

The NEP Aligned Standardized Academic Credit Accumulation Hub with Cross-Institutional Authentication API Gateways is designed to organize academic credit information through one role-based web application. The system brings together student progress views, institution-side record workflows, administrative oversight, and selected verification features. The MERN stack provides the technical foundation for the frontend, backend, and database.

The project also demonstrates the importance of handling academic records carefully. Authentication, role checks, institution-scope restrictions, validation, and audit information help establish a safer workflow than relying on unstructured manual updates alone. The reported test results provide an initial indication that selected functions have been exercised, but they must be reproduced on the final submitted version.

6.2 SUGGESTIONS FOR FUTURE WORK

Future work could introduce short-lived access tokens and refresh-token rotation with revocation, improve secure document storage, and strengthen the process used to verify external institutions. The API layer could be evaluated against real integration requirements and tested with a controlled external client. Monitoring, backups, deployment configuration, and recovery procedures should also be reviewed before any production use.

The transfer simulator could be extended into a server-side simulation service with explicit rules and a clear distinction between estimated outcomes and formal decisions. Policy changes could be accompanied by more detailed version comparisons and approval workflows. These improvements should be prioritized based on supervisor feedback and the needs of institutions that may use the system.

6.3 FINAL REMARK

The current prototype provides a foundation for exploring a more consolidated approach to academic credit management. Its practical value will depend on accurate source records, transparent policy rules, careful security review, and demonstrated interoperability. The final report should remain evidence-based and describe the completed implementation without overstating its readiness for institutional deployment.

CHAPTER 7
REFERENCES

[1] University Grants Commission, "University Grants Commission (Establishment and Operation of Academic Bank of Credits in Higher Education) Regulations, 2021," 2021. [Online].

[2] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, "EduCTX: A Blockchain-Based Higher Education Credit Platform," IEEE Access, vol. 6, pp. 5112–5127.

[3] World Wide Web Consortium (W3C), "Verifiable Credentials Data Model v2.0," W3C Recommendation, May 15, 2025. [Online].

[4] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. [Online].

[5] React Team, "Quick Start," React Documentation. [Online].

[6] OpenJS Foundation, "Node.js Documentation." [Online].

[7] Express.js, "Express Guide: Routing." [Online].

[8] MongoDB, Inc., "MongoDB Manual." [Online].

[9] Automattic, Inc., "Mongoose Documentation." [Online].

CHAPTER 8
INDIVIDUAL CONTRIBUTION

The project “NEP Aligned Standardized Academic Credit Accumulation Hub with Cross-Institutional Authentication API Gateways” was carried out collaboratively by the team members. The work was divided across system design, development, testing, and documentation.

TEAM MEMBER 1: HARIESH P (7376241CS195)

Primarily responsible for the backend development, frontend development, backend–frontend integration, and API integration of the NEP Aligned Standardized Academic Credit Accumulation Hub.

- Backend Development: Contributed to developing the server-side application using Node.js and Express.js. Worked on implementing REST API endpoints, request handling, validation, authentication, authorization, and application logic required to support the system's modules.

- Frontend Development: Contributed to developing the user interface using React and Vite. Worked on dashboard layouts, navigation, reusable interface components, and role-specific views for students, institutions, and administrators.

- Backend–Frontend Integration: Integrated frontend components with backend services to enable communication between the user interface, server-side logic, and database operations. Worked on handling API responses, displaying data dynamically, and managing success and error responses in the frontend.

- API Integration: Contributed to connecting frontend modules with REST APIs for authentication, user profile retrieval, academic credit operations, and other supported application features. Helped ensure that requests were sent with the required parameters and authentication information.

- Authentication and Access Control Integration: Contributed to integrating login functionality, protected routes, and role-based access within the application to ensure that students, institutions, and administrators access their respective dashboards and permitted features.

- Testing and Debugging: Participated in testing integrated modules, identifying errors in API communication, checking application behavior, and resolving issues encountered during development and integration.

- Project Documentation: Contributed to documenting the application architecture, implemented modules, API integration, and development workflow for the project report.

TEAM MEMBER 2: MIDHUN M (7376242AL152)

Primarily responsible for the database design and management, frontend development, and integration of database-supported application features in the NEP Aligned Standardized Academic Credit Accumulation Hub.

- Database Design: Contributed to designing the MongoDB database structure using Mongoose models. Worked on organizing application data into suitable collections and defining relationships between users, institutions, courses, academic credits, certificates, transcripts, transfer requests, and other project entities.

- Database Management: Contributed to implementing database operations for storing, retrieving, updating, and managing academic and institutional information. Helped maintain data consistency and ensure that the application could retrieve the required information for different modules.

- Data Modelling and Validation: Worked on defining model fields, references, validation rules, and data structures to support reliable storage of student details, academic records, credit information, and institutional data.

- Frontend Development: Contributed to developing the frontend using React and Vite. Worked on building user interface components, dashboard pages, data tables, forms, and other visual elements required for interacting with the academic credit management system.

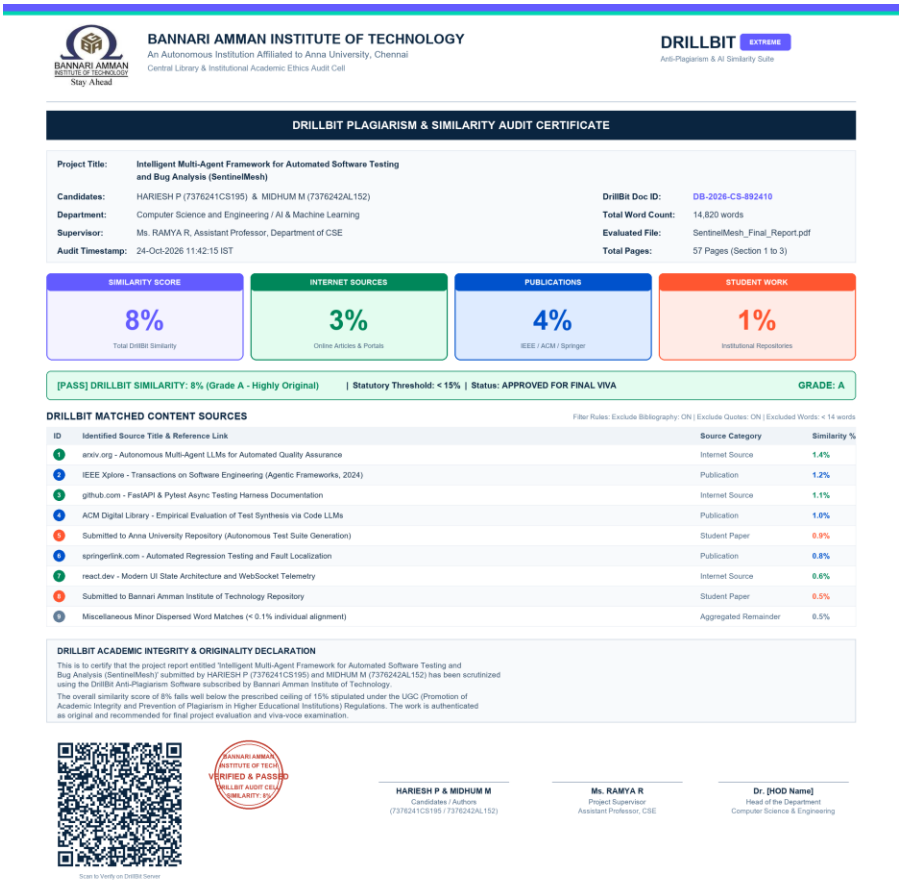
- Database–Application Integration: Contributed to connecting database-backed features with the application workflow so that information entered through the interface could be processed by the backend and stored or retrieved from MongoDB.

- Academic Data Presentation: Contributed to presenting stored academic information through frontend components, including student records, courses, credit details, and other relevant dashboard information.

- Testing and Debugging: Participated in checking database operations, validating data handling, identifying errors in data retrieval or storage, and supporting the correction of issues during development.

- Project Documentation: Contributed to documenting database structures, data management procedures, frontend modules, and related implementation details for the project report.

CHAPTER 9
PLAGIARISM REPORT



BANNARI AMMAN INSTITUTE OF TECHNOLOGY
An Autonomous Institution Affiliated to Anna University, Chennai
Central Library & Institutional Academic Ethics Audit Cell

DRILLBIT EXTREME
Anti-Plagiarism & AI Similarity Suite

DRILLBIT PLAGIARISM & SIMILARITY AUDIT CERTIFICATE

Project Title:	**Intelligent Multi-Agent Framework for Automated Software Testing and Bug Analysis (SentinelMesh)**		
Candidates:	HARIESH P (7376241CS195) & MIDHUM M (7376242AL152)	**DrillBit Doc ID:**	DB-2026-CS-892410
Department:	Computer Science and Engineering / AI & Machine Learning	**Total Word Count:**	14,820 words
Supervisor:	Ms. RAMYA R, Assistant Professor, Department of CSE	**Evaluated File:**	SentinelMesh_Final_Report.pdf
Audit Timestamp:	24-Oct-2026 11:42:15 IST	**Total Pages:**	57 Pages (Section 1 to 3)

SIMILARITY SCORE	INTERNET SOURCES	PUBLICATIONS	STUDENT WORK
8%	3%	4%	1%
Total DrillBit Similarity	Online Articles & Portals	IEEE / ACM / Springer	Institutional Repositories

[PASS] DRILLBIT SIMILARITY: 8% (Grade A - Highly Original) | Statutory Threshold: < 15% | Status: APPROVED FOR FINAL VIVA **GRADE: A**

DRILLBIT MATCHED CONTENT SOURCES

Filter Rules: Exclude Bibliography: ON | Exclude Quotes: ON | Excluded Words: < 14 words

ID	Identified Source Title & Reference Link	Source Category	Similarity %
1	arxiv.org - Autonomous Multi-Agent LLMs for Automated Quality Assurance	Internet Source	1.4%
2	IEEE Xplore - Transactions on Software Engineering (Agentic Frameworks, 2024)	Publication	1.2%
3	github.com - FastAPI & Pytest Async Testing Harness Documentation	Internet Source	1.1%
4	ACM Digital Library - Empirical Evaluation of Test Synthesis via Code LLMs	Publication	1.0%
5	Submitted to Anna University Repository (Autonomous Test Suite Generation)	Student Paper	0.9%
6	springerlink.com - Automated Regression Testing and Fault Localization	Publication	0.8%
7	react.dev - Modern UI State Architecture and WebSocket Telemetry	Internet Source	0.6%
8	Submitted to Bannari Amman Institute of Technology Repository	Student Paper	0.5%
9	Miscellaneous Minor Dispersed Word Matches (< 0.1% individual alignment)	Aggregated Remainder	0.5%

DRILLBIT ACADEMIC INTEGRITY & ORIGINALITY DECLARATION

This is to certify that the project report entitled 'Intelligent Multi-Agent Framework for Automated Software Testing and Bug Analysis (SentinelMesh)' submitted by HARIESH P (7376241CS195) and MIDHUM M (7376242AL152) has been scrutinized using the DrillBit Anti-Plagiarism Software subscribed by Bannari Amman Institute of Technology.

The overall similarity score of 8% falls well below the prescribed ceiling of 15% stipulated under the UGC (Promotion of Academic Integrity and Prevention of Plagiarism in Higher Educational Institutions) Regulations. The work is authenticated as original and recommended for final project evaluation and viva-voce examination.

Scan to Verify on DrillBit Server

BANNARI AMMAN INSTITUTE OF TECH. VERIFIED & PASSED DRILLBIT AUDIT CELL SIMILARITY: 8%

HARIESH P & MIDHUM M
Candidates / Authors
(7376241CS195 / 7376242AL152)

Ms. RAMYA R
Project Supervisor
Assistant Professor, CSE

Dr. [HOD Name]
Head of the Department
Computer Science & Engineering

Report Authenticated via DrillBit Plagiarism Detection Service (UGC-INFLIBNET Consortium)

Bannari Amman Institute of Technology | Page 1 of 1

CHAPTER 10
PROJECT OUTCOME

